

Buffered (Seqüències a través de Buffer)

Índex

Buffered (Seqüències a través de Buffer).....	1
1. Introducció.....	1
2. FileReader & BufferedReader.....	3
3. FileWriter & BufferedWriter.....	5
4. FileInputStream & BufferedInputStream.....	7
5. FileInputStream & InputStreamReader & BufferedReader.....	8
6. FileOutputStream & BufferedOutputStream.....	8

1. Introducció

Realment escriure i llegir un fitxer caràcter a caràcter és lent i poc eficient, per millorar això utilitzarem un **buffer**, una memòria interna entre el programa Java i el fitxer que volem accedir.

Per exemple, si volem llegir un fitxer, el **buffer** s'omplirà amb tota la informació del fitxer, i després serà Java qui accedirà al **buffer** i s'encarregarà de descarregar el contingut. D'aquesta manera tenim una manera d'accedir a fitxers més eficient, és a dir si es desconnecta o es perd la comunicació momentàniament ja tenim el **buffer** amb la informació necessària.

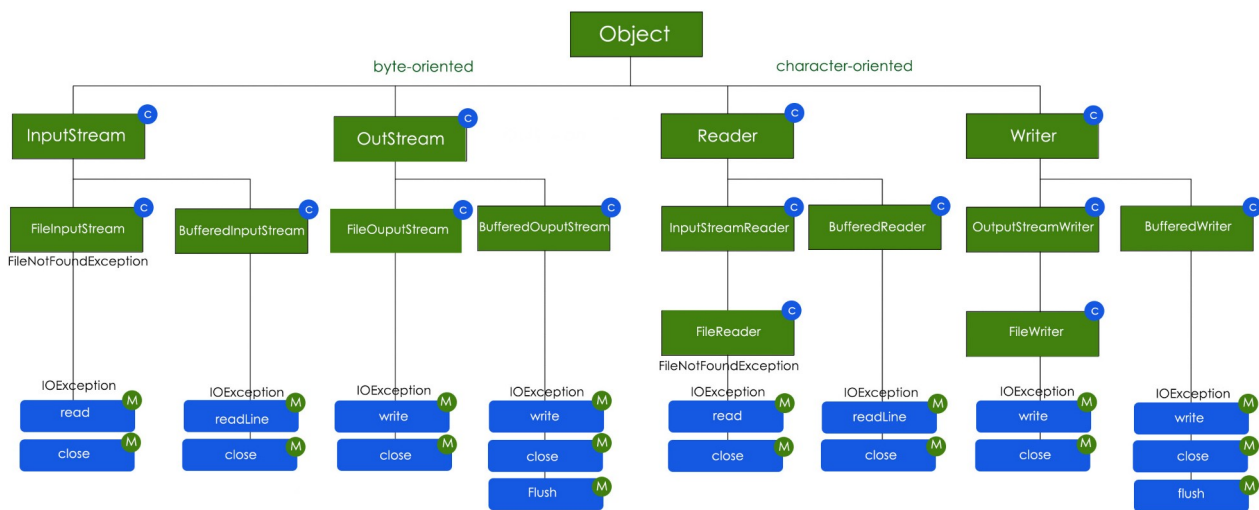
Utilitzarem les classes: **BufferedReader**, **BufferedWriter**, **BufferedInputStream** i **BufferedOutputStream**

Tenim dos mètodes nous:

- **readLine()**: per llegir línies completes amb el mètode **readLines()**. Un dels avantatges de treballar amb aquestes classes, és poder treballar amb línies senceres en comptes d'un caràcter com fèiem amb el mètode **read()**.

- **flush()**: amb aquest mètode ens assegurem que tot el que tenim al **buffer** i encara està pendent de ser traspassat al fitxer, és escrit al fitxer.

Flux de bytes:		Flux de caràcters:	
Per qualsevol tipus de fitxer		Per fitxers de tipus text	
Classes:		Classes:	
FileInputStream	BufferedInputStream	FileReader	BufferedReader
Mètode:	Mètode:	Mètodes:	Mètodes:
close	read close	close	readLine read close
Classe:		Classes:	
FileOutputStream	BufferedOutputStream	FileWriter	BufferedWriter
Mètodes:	Mètodes:	Mètodes:	Mètodes:
close	write flush close	close	write flush close



Hem d'importar el *package java.io.** :

```

import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
  
```

2. FileReader & BufferedReader

Tenim diferents opcions alhora de llegir el contingut d'un fitxer que provenen de diferents classes. Considerem que tenim el següent fitxer de text:

```
FileReader fr = new FileReader("file.txt");
```

Mètode	Exemple	Prové de la classe	Significat
<code>int read()</code>	<code>int c = fr.read()</code>	BufferedReader	Llegeix un caràcter en codi ASCII.
<code>String readLine()</code>	<code>String text = fr.readLine()</code>	BufferedReader	Llegeix una línia sencera de text
<code>read(char[] chars)</code>	<code>char[] chars = new char[10];</code> <code>int totalChar = fr.read(chars)</code>	Reader	Guarda a <code>chars</code> (array de 10 caràcters) els 10 caràcters del fitxer llegits. <code>totalChar</code> conté el nombre de caràcters llegits (en aquest cas 10).
<code>read(char[] chars, int x, int length)</code>	<code>char[] chars = new char[40];</code> <code>int totalChar = fr.read(chars, 10, 30);</code>	BufferedReader	Guarda a <code>chars</code> (array de caràcters) a partir de la posició 10 els 30 caràcters del fitxer llegits. <code>totalChar</code> conté el nombre de caràcters llegits (en aquest cas 30).

Exemple de com podem **llegir un fitxer de text**:

- Instanciem un objecte de la classe **FileReader** que serà el nostre fitxer de text.
- Instanciem també un objecte de la classe **BufferedReader**.
- Degut a que **FileReader** llança una excepció del tipus **FileNotFoundException**, aquesta s'ha de capturar amb un **try-catch**. Aquesta excepció controla si el fitxer no existeix.
- Amb el mètode **readLine()** llegim una línia de text (en comptes de llegir caràcter a caràcter). Degut a què aquest mètode llança una excepció **IOException**, hem de capturar-la amb un **try-catch**. Aquesta excepció controla si el fitxer en el moment de ser llegit no es pot obrir (perquè s'ha traslladat a un altre lloc, o s'ha modificat en el moment de la lectura).
- Anem llegint fins que trobem un contingut nul.
- Tanquem el **buffer** amb el mètode **close()**. Aquest mètode també tanca el **FileReader**.

Exemple 1: Utilitzant mètode **readLine()** de la classe **BufferedReader** que retorna un *String*.

```
try {
    FileReader file = new FileReader("fitxer2.txt");
    BufferedReader myBuffer = new BufferedReader(file);
    String linia = "";
    while (linia != null) {
        linia = myBuffer.readLine();

        System.out.println(linia);
    }
    myBuffer.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

Es pot declarar el **buffer** amb una classe anònima, ja que la variable *file* de tipus **FileReader** no s'utilitza:

En comptes de:

```
FileReader file = new FileReader("fitxer2.txt");
BufferedReader myBuffer = new BufferedReader(file);
```

Podem utilitzar una classe anònima:

```
BufferedReader myBuffer = new BufferedReader(new FileReader("fitxer2.txt"));
```

Exemple 2: Llegim el fitxer caràcter a caràcter.

El mètode **read()** de la classe **BufferedReader** retorna un *int*.

```
try {
    FileReader fr = new FileReader("fitxer1.txt");
    BufferedReader myBuffer = new BufferedReader(fr);
    int c = myBuffer.read(); //reads an ASCII char

    while (c != -1) {
        System.out.print((char) c);
        c = myBuffer.read(); //reads an ASCII char
    }
    myBuffer.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

Exemple 3: Guardem el contingut del fitxer de text en un array de tipus *char*.

El mètode **read(char[])** de la classe **BufferedReader** retorna un *int*.

```
try {
    char[] buffer = new char[1024]; //specify a size

    FileReader fr = new FileReader("fitxer1.txt");
    BufferedReader myBuffer = new BufferedReader(fr);
    while (myBuffer.read(buffer) != -1) {

        System.out.println(new String(buffer));
        buffer = new char[1024];
    }

    myBuffer.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

Exemple 4: Guardem el contingut del fitxer de text en un array de tipus *char* indicant la primera posició i la llargada del buffer.

El mètode **read(char[], int, int)** de la classe **BufferedReader** retorna un *int*. En aquest exemple, llegirà de 10 en 10 i escriurà 100 caràcters en blanc cada vegada.

```
try {
    char[] buffer = new char[1024]; //specify a size

    FileReader fr = new FileReader("fitxer1.txt");
    BufferedReader myBuffer = new BufferedReader(fr);
    while (myBuffer.read(buffer, 100, 10) != -1) {

        System.out.println(new String(buffer));
        buffer = new char[1024];
    }

    myBuffer.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

3. FileWriter & BufferedWriter

Exemple de com podem **escriure en un fitxer de text**:

- Instanciem un objecte de la classe **FileWriter** que serà el nostre fitxer de text. Indiquem **true** si volem escriure al final del fitxer, **false** si volem sobreescriure el contingut del fitxer.
- Instanciem també un objecte de la classe **BufferedWriter**.
- Recorrem un a un els caràcters que volem escriure.
- Amb el mètode **write()** i **flush()** guardem les dades en el fitxer, el mètode **flush()** escriu tot e que queda en el **buffer** que encara no s'ha passat al fitxer (s'ha de realitzar després de **write()** per assegurar-nos que no queda cap dada pendent en el **buffer** per ser escrita al fitxer). Recollim l'excepció **IOException** amb un **try-catch**. Aquesta excepció controla si el fitxer en el moment de ser llegit no es pot obrir (perquè s'ha traslladat a un altre lloc, o s'ha modificat en el moment de la lectura).
- Tanquem el **buffer** amb el mètode **close()**. Aquest també tanca el **FileWriter**.

Per escriure a un fitxer de text farem servir la classe **BufferedWriter**. El mètode **write()** ens escriu contingut en un fitxer, tenim diferents mètodes que provenen de diferents classes i subclasses, podem escriure un caràcter (en format ASCII), una cadena de caràcters o un *String*:

Mètode	Prové de la classe	Significat
void writer(int c)	BufferedWriter	Escriu un caràcter en valor ASCII
void writer(char [] chars)	Writer	Escriu una cadena de caràcters
void writer(char [] chars, int begin, int len)	BufferedWriter	Escriu un tros de la cadena de caràcters
void writer(String s)	Writer	Escriu un <i>String</i>
void writer(String s, int begin, int len)	BufferedWriter	Escriu una part del <i>String</i>

Exemple 1: escrivim en el fitxer un *String*.

```
String text="Escriu això a un fitxer\nTambé això en un fitxer\nI això també.";
try {
    FileWriter file = new FileWriter("fitxer2.txt", true);
    BufferedWriter myBuffer = new BufferedWriter(file);
    myBuffer.write(text);

    myBuffer.flush();

    System.out.println(text);
    myBuffer.close();
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

Exemple 2: escrivim en el fitxer una porció d'un *String*. Des de la posició 10, escrivim els 20 caràcters següents.

```
String text="Escriu això a un fitxer\nTambé això en un fitxer\nI això també.";
try {
    FileWriter file = new FileWriter("fitxer2.txt", true);
    BufferedWriter myBuffer = new BufferedWriter(file);
    myBuffer.write(text, 10, 20);

    myBuffer.flush();

    System.out.println(text);
    myBuffer.close();
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

Output: Escriurà a partir de la posició 10, els 20 caràcters següents.

ò a un fitxer
També

Exemple 3: escrivim en el fitxer caràcters en ASCII.

```
try {
    FileWriter file = new FileWriter("fitxer3.txt", false);
    BufferedWriter myBuffer = new BufferedWriter(file);
    myBuffer.write((int)'A');
    myBuffer.write((int)'B');
    myBuffer.write((int) '?');
    myBuffer.newLine();
    myBuffer.write((int) '%');
    myBuffer.write((int) 'C');
    myBuffer.write((int) '*');
    myBuffer.flush(); // guardar dades al fitxer
    myBuffer.close();
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

← Insereix una línia nova

Output: La sortida del fitxer contindrà el següent:

```
AB?
%C*
```

Exemple 4: escrivim en el fitxer una cadena de caràcters. Des de la posició 5, escrivim els 50 caràcters següents.

```
String text="Escriu això a un fitxer\nTambé això en un fitxer\nI això també.";
try {
    FileWriter file = new FileWriter("fitxer3.txt", false);
    BufferedWriter myBuffer = new BufferedWriter(file);
    char[] buffer = text.toCharArray();
    myBuffer.write(buffer, 5, 50);

    myBuffer.flush(); // guardar dades al fitxer

    myBuffer.close();
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

Output: La sortida del fitxer contindrà el contingut des de la posició 5, els 50 caràcters següents:

```
u això a un fitxer
També això en un fitxer
I això
```

4. FileInputStream & BufferedInputStream

Exemple de com podem **llegir un fitxer de bytes**:

- Podem definir la mida del **buffer**, un valor acceptable perquè funcioni a la majoria de sistemes és 8*1024. (opcional)
- Instanciem un objecte de la classe **FileInputStream** que serà el nostre fitxer de text.
- Instanciem un objecte de la classe **BufferedInputStream**, podem especificar la mida del **buffer** (opcional).
- Recorrem el fitxer fins trobar el final de fitxer (-1).
- Amb el mètode **read()** llegim el contingut del **buffer**.
- Recollim les excepcions **FileNotFoundException** i **IOException**.
- Tanquem el **buffer** amb el mètode **close()**. Aquest també tanca el **FileInputStream**.

Codi exemple, opció sense especificar la mida del buffer:

```
try {
    FileInputStream file = new FileInputStream("drawing.jpg");
    BufferedInputStream myBuffer = new BufferedInputStream(file);
    int data = myBuffer.read();
    while (data != -1) {
        System.out.println(data);
        data = myBuffer.read();
    }
    myBuffer.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

Opció especificant la mida del buffer:

```
int bufferSize = 8 * 1024; //opcional, recommended size
try {
    FileInputStream file = new FileInputStream("drawing.jpg");
    BufferedInputStream myBuffer = new BufferedInputStream(file, bufferSize);
    int data = myBuffer.read();
    while (data != -1) {
        System.out.println(data);
        data = myBuffer.read();
    }
    myBuffer.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    //In case the file has been moved, delete, ...
    System.out.println("Error: " + e.getMessage());
}
```

5. FileInputStream & InputStreamReader & BufferedReader

Per **llegir fitxers de text amb caràcters especials UTF-8**, haurem de fer servir la classe **Charset** per especificar el tipus de codi i indicar-li a la classe **InputStreamReader** quin Charset farem servir.

Codi exemple:

```
try {
    Charset inputCharset = Charset.forName("UTF-8");

    FileInputStream fis = new FileInputStream("fileUTF8.txt");
    InputStreamReader isr = new InputStreamReader(fis, inputCharset);

    BufferedReader myBuffer = new BufferedReader(isr);
    String line = "";
    line = myBuffer.readLine(); //reads a line of text
    while (line != null) {
        System.out.println(line);
        line = myBuffer.readLine();
    }
    myBuffer.close();
} catch (FileNotFoundException e) {
    System.out.println("File Not Found: " + e.getMessage());
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

6. FileOutputStream & BufferedOutputStream

Exemple de com podem **copiar el contingut d'un fitxer en un altre fitxer**:

- Podem definir la mida del **buffer**, una mida de 8*1024 és la més recomanable.
- Instanciem un objecte de la classe **FileOutputStream** que serà el nostre fitxer de text.
- Instanciem un objecte de la classe **BufferedOutputStream**.
- Llegim les dades d'un fitxer de bytes, i mentre hi hagi dades, escrivim al **buffer**.
- Recollim l'excepció **IOException** i **FileNotFoundException** amb un **try-catch**.
- Tanquem el **buffer** amb **flush()** el mètode **close()**. El **flush** escriu qualsevol cosa que encara hi hagi al **buffer**.

Codi exemple: copiem el fitxer *s.jpg* a *s2.jpg*

```
byte[] buffer = new byte[8192]; // Try to read 8K at a time
int bytesRead;
try {
    FileInputStream fileInput = new FileInputStream("s.jpg");
    FileOutputStream fileOutput = new FileOutputStream("s2.jpg");
    BufferedOutputStream myBuffer = new BufferedOutputStream(fileOutput);
    bytesRead = fileInput.read(buffer);
    while (bytesRead > 0) {
        myBuffer.write(buffer, 0, bytesRead);
        bytesRead = fileInput.read(buffer);
    }
    myBuffer.flush();
    myBuffer.close();
} catch (FileNotFoundException e) {
    System.out.println("File not found: " + e.getMessage());
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
```

Llicència

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is in <http://www.gnu.org/licenses/fdl.txt>